Incident Response Lab Guide

Cybersecurity Masterclass by Joshua Moses | CyberTrace Foundation · Oct 23, 2026 · @Joshua Moses

Lab overview

By the end of this lab you will have built a small, self-contained virtual network, attacked it, detected the attack, and worked a simulated intrusion through all six phases of incident response. You will leave with a SIEM dashboard full of real alerts you generated yourself, forensic evidence you pulled from a live endpoint, and a written post-incident review. That is the whole point: incident response is a skill you build by doing, not by reading a runbook.

The scenario is a credential-theft intrusion on a Windows workstation that leads to lateral movement and attempted persistence. You play the defender. A separate attacker machine runs the offensive steps so the detections you see are genuine, not simulated screenshots.

Who this is for. Students who have finished the virtualization lab (or who can already build and run virtual machines) and are comfortable at a Linux and Windows command line. You do not need prior SIEM or forensics experience. Every command is given in full.

Time. Plan on 6 to 10 hours total. The build (Parts A through E) takes 3 to 4 hours and only has to be done once. The six exercises take 3 to 5 hours and can be split across sessions, because the lab persists between reboots.

What you need.

- A host machine with at least 16 GB RAM and 150 GB free disk. 32 GB RAM makes everything smoother.
- A hypervisor you already know (VMware Workstation, VirtualBox, Fusion, or UTM). The virtualization lab covers installing these.
- Internet access from the VMs during the build, to pull packages and tools.

Safety rules, read these before you start.

1. Everything in this lab stays on an isolated host-only or internal network. Nothing here touches the internet except the package downloads during the build, and nothing here touches a network you do not own.

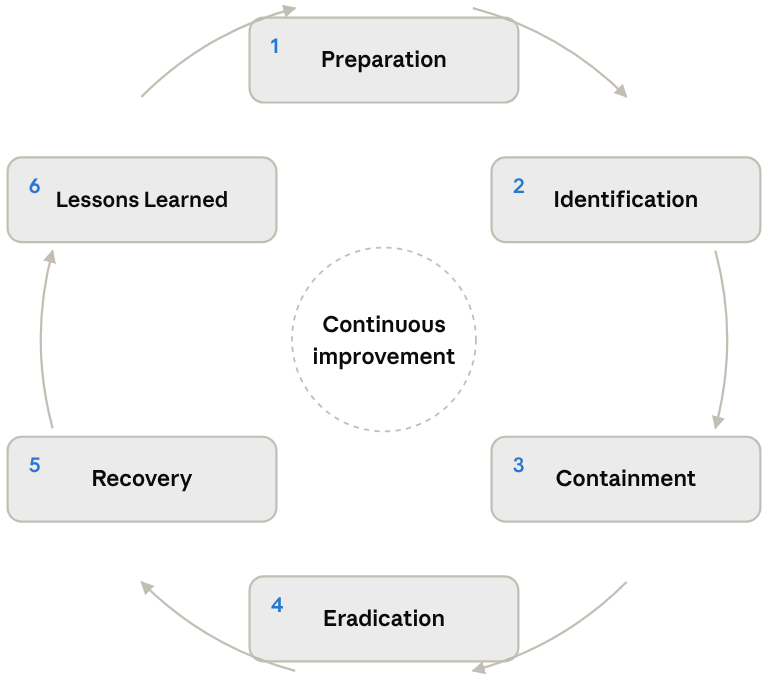
2. You will download and run real offensive tooling (Atomic Red Team, and optionally Mimikatz). Treat these VMs as disposable and infected. Never copy these tools onto a production or personal machine.
3. The attacker techniques are run only against your own lab VMs. Running them anywhere else is illegal and is not what this course is about.
4. Snapshot every VM at the "clean build" stage before you start the exercises. You will roll back to those snapshots in the Recovery exercise, and any time you want to run the lab again.

The incident response lifecycle

Incident response is a loop, not a line. You prepare, you detect, you contain, you clean up, you recover, and then you feed what you learned back into preparation so the next incident goes better. The classic six-phase model below (the SANS / traditional model) is what this lab walks you through, phase by phase.

Incident response is a loop, not a line

Each phase feeds the next; lessons learned feed back into preparation



incident response lifecycle · 6 phases, 1 feedback loop

1. **Preparation.** Build the capability before anything happens: policies, a team with defined roles, tooling (SIEM, EDR, logging), severity levels, and tested backups.

2. **Identification.** Decide whether suspicious activity is actually an incident. Watch alerts and telemetry, find indicators of compromise, build a timeline, and classify severity.
3. **Containment.** Stop the bleeding. Isolate hosts, cut malicious connections, disable accounts, and preserve evidence before you take destructive action.
4. **Eradication.** Remove the attacker and the root cause: malware, persistence, exploited vulnerabilities, rogue accounts, and compromised credentials.
5. **Recovery.** Bring systems back safely from clean backups, validate they work, and watch for the attacker coming back.
6. **Lessons Learned.** Run the post-incident review, find the root cause, fix the gaps, and update playbooks and training.

How this maps to NIST. NIST SP 800-61 Revision 3 reframes incident response so it lives inside the broader cybersecurity risk-management cycle of NIST CSF 2.0 rather than standing alone as six steps. The six-phase model still describes the work; CSF 2.0 describes where that work sits in the organization. The mapping below lines them up so you can speak both languages.

CSF 2.0 function	What it covers	Six-phase phase it carries
Govern	Policies, responsibilities, risk oversight	Preparation
Identify	Assets, vulnerabilities, organizational risk	Preparation
Protect	Safeguards and response readiness	Preparation
Detect	Finding and analyzing possible incidents	Identification
Respond	Managing, analyzing, containing, mitigating	Containment, Eradication
Recover	Restoring affected assets and operations	Recovery, Lessons Learned

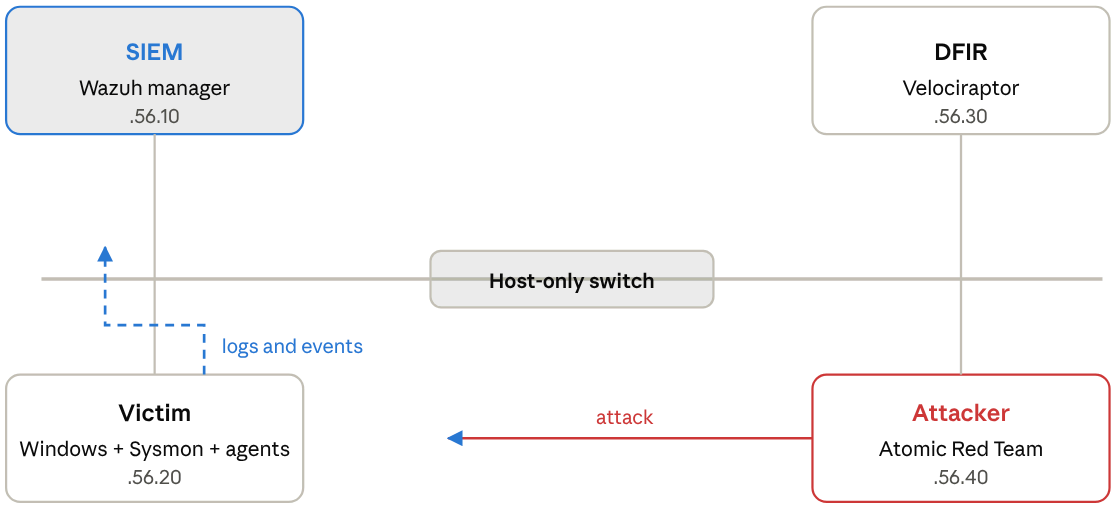
The key difference: the six-phase model treats improvement as a final step, while CSF 2.0 and SP 800-61r3 treat continuous improvement as something that runs through the whole lifecycle. In this lab you will feel that directly, because the detections you tune in Preparation are the ones that fire in Identification. Reference: NIST SP 800-61 Rev. 3.

Lab architecture and software

The lab is four virtual machines on one isolated network. One box runs the SIEM, one is the victim endpoint you will defend, one is the forensics server, and one is the attacker. In a real SOC these would be dozens of separate systems; here you run the whole stack yourself so you see how every piece connects.

Four VMs on one isolated host-only network

192.168.56.0/24 · nothing routes to the internet or any real network



lab network · 4 VMs, host-only

VM	Role	OS	Specs	Lab IP
SIEM	Wazuh manager, indexer, dashboard	Ubuntu Server 22.04 or 24.04	4 vCPU, 8 GB RAM, 50 GB disk	192.168.56.10
Victim	Monitored Windows endpoint	Windows 10 or 11	2 vCPU, 4 GB RAM, 60 GB disk	192.168.56.20
DFIR	Velociraptor forensics server	Ubuntu Server 22.04 or 24.04	2 vCPU, 4 GB RAM, 40 GB disk	192.168.56.30
Attacker	Adversary simulation	Kali Linux or the Victim itself	2 vCPU, 4 GB RAM, 40 GB disk	192.168.56.40

The IPs above are the examples used throughout this guide. If your hypervisor hands out a different host-only subnet, use your own addresses and keep them consistent.

The software, and why each tool is here.

- **Wazuh** is the SIEM and XDR platform. It is the heart of the lab. The agent on the victim ships logs and security events to the manager, the manager runs them through its ruleset and maps them to MITRE ATT&CK, and the dashboard is where you will watch alerts fire and investigate them. Free, open source, and the same platform used in real small-to-mid SOCs. This lab uses Wazuh 4.14.
- **Sysmon** (System Monitor, from Microsoft Sysinternals) is a Windows driver that logs rich detail the default Windows logs miss: full process creation with command lines and hashes, network connections, and more. Without Sysmon your detections are blind to most of what an attacker actually does. Wazuh reads the Sysmon event channel.
- **Velociraptor** is the digital forensics and incident response (DFIR) tool. Where Wazuh tells you something happened, Velociraptor lets you reach into the live endpoint and collect evidence: running processes, autoruns, artifacts, files. You use it in the Identification and Eradication exercises. This lab uses Velociraptor 0.77.3.
- **Atomic Red Team** is a library of small, documented attack tests mapped to MITRE ATT&CK. It is how you generate real malicious activity safely, so the alerts you investigate are genuine. Run by the attacker.
- **Mimikatz** (optional) is the classic credential-dumping tool. It makes the credential-theft part of the scenario concrete. It is flagged by every EDR on earth, which is exactly why it is a good detection test. Only ever run it inside this lab.

Everything here is free. The only account you may need is a Microsoft evaluation download for the Windows VM, which the virtualization lab already covers.

Part A: build the lab network and VMs

This part gets four VMs onto one isolated network. If you did the virtualization lab, the VM creation steps will be familiar; the new piece is wiring them to a private network with no route to the outside.

A1. Create the isolated network. The goal is a host-only or internal network so the lab VMs can talk to each other and to your host, but not to the internet or your real LAN during the exercises. Do this once in your hypervisor:

- **VirtualBox:** File > Tools > Network Manager > Host-only Networks > Create. Note the subnet it assigns (often 192.168.56.0/24). Leave the DHCP server enabled or assign static IPs on each VM.

- **VMware Workstation / Fusion:** Edit > Virtual Network Editor (Workstation) or Settings > Network (Fusion). Add a network, set it to Host-only, and note the subnet.
- **UTM:** In each VM's network settings choose the Host Only or Emulated VLAN mode so the guests share one private segment.

For the build steps only, you may temporarily attach a second NAT adapter to each VM so it can download packages and tools. Remove or disconnect that NAT adapter before you start the exercises. From then on the lab is air-gapped.

A2. Build the SIEM VM. Create a VM with 4 vCPU, 8 GB RAM, 50 GB disk and install Ubuntu Server 22.04 or 24.04. Give it a static address on the lab network:

```
sudo nano /etc/netplan/50-cloud-init.yaml
```

Set the host-only interface (name it with `ip a`, often `enp0s8`) to a static IP, then apply:

```
network:
  version: 2
  ethernets:
    enp0s8:
      addresses: [192.168.56.10/24]
```

```
sudo netplan apply
```

A3. Build the DFIR VM. Same steps as the SIEM VM but 4 GB RAM and 40 GB disk, address 192.168.56.30.

A4. Build the Victim VM. Create a VM with 2 vCPU, 4 GB RAM, 60 GB disk and install Windows 10 or 11. Use a Microsoft evaluation image if you do not have a license. Set a static IP of 192.168.56.20 in Settings > Network > your adapter > IP assignment > Manual. Create a local admin account you will use throughout, for example `labadmin`.

A5. Build the Attacker VM. Simplest option: install Kali Linux (2 vCPU, 4 GB RAM) at 192.168.56.40. If you want to keep the lab to three VMs, you can instead run the Atomic Red Team tests directly on the Victim; the detections still work because Sysmon and the Wazuh agent see the activity either way. The guide notes where the single-VM path differs.

A6. Verify connectivity. From the SIEM VM, confirm it can reach the Victim and DFIR hosts, and confirm the Victim can reach the SIEM:

```
ping -c 3 192.168.56.20
ping -c 3 192.168.56.30
```

From the Victim (PowerShell): `Test-NetConnection 192.168.56.10 -Port 1514`. You want every host reachable on the lab subnet and nothing else.

A7. Snapshot nothing yet. You will snapshot after the full build, at the end of Part E. For now, keep going.

Part B: install and configure Wazuh (the SIEM)

All of this runs on the SIEM VM (192.168.56.10). The Wazuh installation assistant puts the manager, indexer, and dashboard on one host in a single run, which is exactly right for a lab. Reference: Wazuh quickstart.

B1. Update the OS and install helpers.

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y curl gnupg apt-transport-https lsb-release
```

B2. Download the installation assistant.

```
cd ~
curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh
```

B3. Run the all-in-one install. The `-a` flag installs all four components (manager, indexer, dashboard, Filebeat) on this host. Add `-i` to skip the hardware check if the installer complains about your lab specs. The run takes 10 to 15 minutes; let it finish without interrupting.

```
sudo bash wazuh-install.sh -a -i 2>&1 | tee ~/wazuh-install.log
```

B4. Capture the admin password. At the end the installer prints the generated admin password once. Copy it immediately. If you miss it, pull it back out of the bundle the installer leaves behind:

```
sudo tar -O -xf wazuh-install-files.tar wazuh-install-files/wazuh-
passwords.txt | grep -A1 "'admin'"
```

B5. Confirm the services are running.

```
sudo systemctl is-active wazuh-manager wazuh-indexer wazuh-dashboard filebeat
```

All four should print `active`. Check the indexer cluster is healthy (use your admin password):

```
curl -sk -u admin:'YOUR_PASSWORD' https://localhost:9200/_cluster/health?
pretty
```

You want `"status" : "green"`.

B6. Log in to the dashboard. From your host browser, go to `https://192.168.56.10` (note HTTPS). The browser will warn about the self-signed certificate; that is expected in a lab, accept it and continue. Log in as `admin` with the password from B4. You now have an empty SIEM waiting for agents.

B7. Rotate the admin password (good hygiene). Optional but worth doing, since you now know where the password lives. This is itself a Preparation-phase habit.

```
sudo /usr/share/wazuh-indexer/plugins/opensearch-security/tools/wazuh-
passwords-tool.sh --change-all
```

If anything fails here, jump to the Troubleshooting section before moving on. A broken SIEM makes every later exercise impossible.

Part C: the Windows endpoint (Sysmon + Wazuh agent)

All of this runs on the Victim VM (192.168.56.20). Open PowerShell as Administrator for every command. This is where the endpoint starts talking to the SIEM.

C1. Install Sysmon with a good config. Sysmon ships with almost no logging by default, so you install it with a tuned configuration. Download the Sysinternals Sysmon binary and the Wazuh-provided config:

```
cd $env:USERPROFILE\Downloads
Invoke-WebRequest -Uri https://download.sysinternals.com/files/Sysmon.zip -
OutFile Sysmon.zip
Expand-Archive Sysmon.zip -DestinationPath Sysmon
Invoke-WebRequest -Uri https://wazuh.com/resources/blog/emulation-of-attack-
techniques-and-detection-with-wazuh/sysmonconfig.xml -OutFile
Sysmon\sysmonconfig.xml
```

Install and start Sysmon with that config (accept the EULA automatically):

```
cd Sysmon
.\Sysmon64.exe -accepteula -i sysmonconfig.xml
```

Confirm it is logging: open Event Viewer and look under Applications and Services Logs > Microsoft > Windows > Sysmon > Operational. You should see events appearing.

C2. Install the Wazuh agent. Download the agent MSI and install it pointed at your SIEM. The `WAZUH_MANAGER` value is the install-time way to register the agent to the manager:

```
cd $env:USERPROFILE\Downloads
Invoke-WebRequest -Uri https://packages.wazuh.com/4.x/windows/wazuh-agent-
4.14.0-1.msi -OutFile wazuh-agent.msi
msiexec.exe /i wazuh-agent.msi /q WAZUH_MANAGER="192.168.56.10"
WAZUH_AGENT_NAME="victim-win"
```

C3. Tell the agent to read the Sysmon channel. Open the agent config at `C:\Program Files (x86)\ossec-agent\ossec.conf` in an elevated editor (`notepad "C:\Program Files (x86)\ossec-agent\ossec.conf"`). Inside the `<ossec_config>` block, add:

```
<localfile>
  <location>Microsoft-Windows-Sysmon/Operational</location>
  <log_format>eventchannel</log_format>
</localfile>
```

C4. Start the agent.

```
Restart-Service -Name wazuh
Get-Service -Name wazuh
```

The service should show `Running`.

C5. Confirm the agent is connected. On the SIEM dashboard, go to the Agents (or Endpoints) view. The `victim-win` agent should appear with status **Active**. If it shows Disconnected or Never connected, check that the Victim can reach the manager on TCP 1514 (the `Test-NetConnection` from Part A) and that no host firewall is blocking it.

C6. Generate a first test event. On the Victim, run `whoami`. Within a few seconds, on the dashboard go to Threat Hunting / Discover, filter for the agent, and you should see the process-creation event. That single event proves the whole pipeline works: Sysmon logged it, the agent shipped it, the manager processed it, and the dashboard shows it. That is your detection pipeline end to end.

If you are running the single-VM path (no separate Kali box), the Victim is also your attacker. Nothing changes here; you just run the Part E attack steps on this same machine.

Part D: Velociraptor (the DFIR server)

Wazuh tells you something happened. Velociraptor lets you go look. In this part you stand up the Velociraptor server on the DFIR VM (192.168.56.30) and enroll the Victim as a client so that, during the incident, you can pull live forensic artifacts from it on demand.

Velociraptor is a single static binary with no database to set up. Reference: Velociraptor releases.

D1. Download the binary (on the DFIR VM).

```
cd ~
wget
https://github.com/Velocidex/velociraptor/releases/download/v0.77.3/velocirap
tor-v0.77.3-linux-amd64
chmod +x velociraptor-v0.77.3-linux-amd64
sudo mv velociraptor-v0.77.3-linux-amd64 /usr/local/bin/velociraptor
velociraptor version
```

D2. Generate the server config. The interactive generator asks a few questions. Choose a self-signed SSL deployment, set the server to listen on the DFIR VM's lab IP, and set the GUI to bind to 0.0.0.0 so you can reach it from your host browser.

```
velociraptor config generate -i
```

When prompted: self-signed SSL; public DNS name `192.168.56.30`; GUI bind address `0.0.0.0`; create a GUI admin user when asked (or do it in D4). It writes `server.config.yaml`.

D3. Start the server.

```
sudo velociraptor --config server.config.yaml frontend -v
```

Leave this running (or install it as a service with `velociraptor --config server.config.yaml service install`).

D4. Create a GUI login (if you did not in D2).

```
velociraptor --config server.config.yaml user add labadmin --role
administrator
```

Browse to `https://192.168.56.30:8889`, accept the self-signed certificate, and log in.

D5. Build the Windows client installer. From the Velociraptor GUI, go to the Server Artifacts / Client configuration area and generate a client MSI, or build the client config on the server:

```
velociraptor --config server.config.yaml config client > client.config.yaml
```

Copy `client.config.yaml` to the Victim (over the lab network, or paste its contents).

D6. Enroll the Victim (on the Victim VM, PowerShell as Admin). Download the Windows Velociraptor binary, then install it as a service with your client config:

```
cd $env:USERPROFILE\Downloads
Invoke-WebRequest -Uri
https://github.com/Velocidex/velociraptor/releases/download/v0.77.3/velocirap
tor-v0.77.3-windows-amd64.exe -OutFile velociraptor.exe
.\velociraptor.exe --config client.config.yaml service install
```

D7. Confirm the client checked in. Back in the Velociraptor GUI, search for clients. The Victim should appear. Click it, then try a quick collection (for example the `Generic.System.Pstree` artifact) to confirm you can pull live data. You now have a forensics capability pointed at the endpoint, ready for the incident.

Part E: the attacker (Atomic Red Team)

The attacker generates the real malicious activity you will investigate. Atomic Red Team is a library of small tests, each one mapped to a MITRE ATT&CK technique, so you know exactly what you are triggering and why it should alert. It runs best on the Victim itself (so Sysmon sees it directly), driven either from the Victim or launched from your Kali attacker box against the Victim.

Reminder: these are real attack techniques. Only ever run them inside this lab.

E1. Install Atomic Red Team on the Victim (PowerShell as Admin).

```
Set-ExecutionPolicy Bypass -Scope Process -Force
IEX (IWR 'https://raw.githubusercontent.com/redcanaryco/invoke-
atomicredteam/master/install-atomicredteam.ps1' -UseBasicParsing)
Install-AtomicRedTeam -getAtomics
```

E2. Confirm it loaded.

```
Import-Module "C:\AtomicRedTeam\invoke-atomicredteam\Invoke-
AtomicRedTeam.psd1" -Force
Invoke-AtomicTest T1033 -ShowDetailsBrief
```

`T1033` is a harmless system-owner discovery test; seeing its detail confirms the framework works.

E3. (Optional) Stage Mimikatz for the credential-theft step. This makes the scenario concrete. Download it onto the Victim, into a known folder, but do not run it yet; you will run it during the exercises so the detection is part of the story.

```
cd $env:USERPROFILE\Downloads
Invoke-WebRequest -Uri
https://github.com/gentilkiwi/mimikatz/releases/latest/download/mimikatz_trun
k.zip -OutFile mimikatz.zip
Expand-Archive mimikatz.zip -DestinationPath mimikatz
```

Your EDR or Windows Defender may quarantine it on download. That is a legitimate detection and fine for the lab; if you want it to survive, add a temporary Defender exclusion for the Downloads folder, and remember you did (you will note it in the Lessons Learned exercise).

E4. Snapshot everything now. This is the single most important build step. With all four VMs built, configured, and verified, power them to a clean state and take a snapshot of each one, named `clean-build`. You will roll the Victim back to this snapshot in the Recovery exercise, and back to it any time you want to run the whole lab again.

- VirtualBox: select the VM > Snapshots > Take.
- VMware: VM menu > Snapshot > Take Snapshot.
- UTM: the save/snapshot control in the VM toolbar.

The build is done. Everything from here is incident response.

Exercise 1: Preparation

Phase goal: build the response capability before the incident. In the real world this is policies, a team, tooling, and tested backups. In this lab you have already built the tooling; now you make it ready to respond and you set the baseline you will measure the incident against.

What you will do

1. **Write a one-page mini incident response plan.** Use the template in the Appendix. Fill in: who does what (in a solo lab you hold every role, so just name them), how you classify severity, and who you would notify. This is the artifact that makes the difference between panic and process.
2. **Confirm your detections are live.** On the Victim, run three Atomic tests and confirm each produces an alert in Wazuh:

   ```
   Invoke-AtomicTest T1033
   Invoke-AtomicTest T1059.001
   Invoke-AtomicTest T1087.001
   ```

   In the dashboard, open Threat Hunting and confirm you see events for each. If T1059.001 (PowerShell) does not alert, your Sysmon or agent config from Part C needs another look. Fixing it now, in Preparation, is the whole lesson.
3. **Build a baseline.** Note what normal looks like: which processes run, how many events per minute the Victim generates at idle. You cannot spot abnormal without knowing normal.
4. **Verify your backup.** Confirm the `clean-build` snapshot exists for the Victim. That snapshot is your tested, known-good backup for the Recovery phase.

Intended result and why. At the end of Preparation you have a written plan, confirmed working detections, a baseline of normal activity, and a verified backup. This matters because every later phase depends on it: you cannot identify an incident if detections are broken, you cannot recover if your backup is untested, and you cannot run an orderly response if no one knows their role. Teams that skip Preparation discover the gaps in the middle of a real incident, which is the worst possible time.

Deliverable: your one-page IR plan and a short note confirming the three detections fired.

Exercise 2: Identification

Phase goal: decide whether suspicious activity is a real incident, then scope it. This is the detective work: alerts, telemetry, indicators of compromise, timeline, severity.

Set the scene. You are the on-call analyst. You will now run the attack chain that represents the intrusion, then switch hats and investigate it as if you did not know what happened. Run these on the Victim, in order, pausing a few seconds between each:

```
Invoke-AtomicTest T1059.001    # malicious PowerShell
Invoke-AtomicTest T1087.001    # local account discovery
Invoke-AtomicTest T1003.001    # LSASS credential access (the theft)
Invoke-AtomicTest T1053.005    # scheduled task (persistence attempt)
```

If you staged Mimikatz, also run it now from its folder to make the credential theft unmistakable: `.\mimikatz\x64\mimikatz.exe "privilege::debug" "sekurlsa::logonpasswords" exit`.

Now investigate (as the analyst)

1. **Triage the alerts.** In the Wazuh dashboard, open the alerts view and sort by severity (rule level). Look for the high-level alerts the chain produced: PowerShell execution, LSASS access, scheduled task creation.
2. **Map to ATT&CK.** Wazuh tags alerts with MITRE technique IDs. Note which techniques fired. You should recognize the chain: execution, then discovery, then credential access, then persistence. That sequence is the story of the intrusion.
3. **Find the indicators of compromise (IOCs).** From the events, pull out: the process names and command lines, the user account involved, the parent process, file paths, and timestamps. These are your IOCs.
4. **Build the timeline.** Order the events by time. First PowerShell ran, then account discovery, then the credential dump, then the scheduled task. Write it down with timestamps. Use the Appendix incident log template.
5. **Pull live evidence with Velociraptor.** Go to the Velociraptor GUI, select the Victim, and collect: process tree (`Generic.System.Pstree`), scheduled tasks, and autoruns. This confirms what the SIEM alerts suggested and preserves forensic evidence.
6. **Classify severity.** Using your Preparation plan's severity scale, rate this incident. Credential theft plus a persistence attempt on an endpoint is high severity: the attacker now has credentials and a way back in.

Intended result and why. You should conclude that this is a confirmed, high-severity incident: a credential-theft intrusion with attempted persistence, with a documented timeline and IOC list. This matters because everything downstream depends on scoping. Contain too little and the attacker stays in; contain too much and you take down systems that were never affected. A solid identification tells you exactly what to contain and what to leave alone.

Deliverable: a completed incident log with timeline, IOC list, affected account and host, ATT&CK techniques, and a severity rating.

Exercise 3: Containment

Phase goal: stop the attacker from doing more damage, while preserving evidence. Containment can be immediate or staged depending on the threat, the operational impact, and what evidence you still need to collect.

Decision first. Before you touch anything, decide your containment strategy and write down why. You have a choice: pull the plug now (fast, but you may lose volatile evidence and tip off the attacker) or collect volatile evidence first, then isolate (slower, better forensics). For this incident, with credentials already stolen, isolating quickly matters, but you will grab volatile data first because you have Velociraptor ready.

What you will do

1. **Preserve volatile evidence (do this before isolating).** In Velociraptor, collect from the Victim: running processes with command lines, network connections, logged-on sessions, and the contents of the scheduled-task and autoruns artifacts. This is the evidence that disappears when you isolate or reboot.
2. **Isolate the endpoint.** Cut the Victim off from the rest of the lab network. The cleanest lab method is to disconnect its network adapter in the hypervisor, or switch it to a "host-only, isolated" segment. In a real environment you would use your EDR's network-isolation feature or a switch ACL. Note the time you isolated.
3. **Disable the compromised account.** The attacker dumped credentials, so the account they touched is burned. On the Victim: `Disable-LocalUser -Name "labadmin"` (or whichever account the incident involved). In a domain you would disable the account at the domain controller and revoke sessions.
4. **Neutralize the persistence you found, carefully.** You found a scheduled task in Identification. Disable it rather than delete it yet, so it stays as evidence: `Disable-ScheduledTask -TaskName "<the malicious task>"`. Full removal happens in

Eradication.

5. **Record every action.** Use the Appendix containment log. Every action, with a timestamp and who did it. This log becomes part of the evidence record and the post-incident review.

Intended result and why. The attacker can no longer reach the endpoint or use the stolen account, the known persistence is inert, and you still hold the volatile evidence. This matters because containment is where you trade speed against evidence. Act too fast and you destroy the proof you need to understand the attack; act too slow and the attacker spreads. The sequence here, evidence then isolation then account disable, is the balance most IR teams strike.

Deliverable: a containment action log and an updated incident scope reflecting what you isolated and disabled.

Exercise 4: Eradication

Phase goal: remove the attacker's access and the underlying cause, so that when you bring the system back it does not get reinfected. Clean up the malware, the persistence, the exploited weakness, and the compromised credentials.

What you will do

1. **Remove persistence for good.** In Containment you disabled the malicious scheduled task; now delete it: `Unregister-ScheduledTask -TaskName "<the malicious task>" -Confirm:$false`. Then use Velociraptor's autoruns artifact again to confirm no other persistence survived (run keys, services, startup folder). The attacker chain only created a scheduled task, but a real intrusion often plants several mechanisms, so you check all of them.
2. **Remove attacker tooling and artifacts.** Delete the files the attacker dropped or used, based on your IOC list from Identification. In this lab that includes the Mimikatz folder and anything the Atomic tests wrote.
3. **Rotate the compromised credentials.** This is the step people forget. The attacker dumped credentials, so changing the password of every account that was exposed is mandatory, even after you disabled the account. In the lab, reset the password for `labadmin` (and any other account in scope).
4. **Address the entry point.** Ask how the attacker got execution in the first place and close it. In this scenario it was unrestricted PowerShell and an over-privileged local account. Mitigations: set PowerShell execution policy and enable Constrained

Language Mode, remove unnecessary local admin rights, and ensure Windows and Defender are fully patched and enabled.

5. **Decide: clean in place or rebuild?** For a single confirmed persistence mechanism you removed and verified, cleaning in place is defensible. If you had any doubt that you found everything, the safe answer is always rebuild from a trusted image. State your decision and your reasoning in the log. For this exercise, document that you would rebuild in a real high-severity credential-theft case, and note why.
6. **Validate eradication.** Re-run the discovery and persistence Atomic tests and confirm your new mitigations block or alert on them. If PowerShell is now constrained, T1059.001 should behave differently. Verified removal is the point: you do not get to claim eradication because you deleted a file, you claim it because you checked the attacker's paths are closed.

Intended result and why. No attacker persistence remains, the tooling is gone, exposed credentials are rotated, and the entry point is closed and verified. This matters because recovery without eradication just hands the system back to the attacker. The credential-rotation and entry-point steps are what separate real eradication from a cosmetic cleanup.

Deliverable: an eradication report listing what you removed, what you rotated, what you hardened, and your verification results.

Exercise 5: Recovery

Phase goal: bring the system and operations back safely, in controlled stages, and watch for the attacker returning. Recovery is where you restore service, but carefully, not all at once.

What you will do

1. **Restore from known-good.** This is where the `clean-build` snapshot earns its keep. Roll the Victim back to that snapshot. In a real environment you would restore from a verified clean backup taken before the compromise. Because the snapshot predates the intrusion, restoring it removes everything the attacker did in one move, which is exactly why a tested backup is the strongest recovery tool you have.
2. **Re-apply the hardening from Eradication.** The clean snapshot predates your fixes too, so re-apply them: the PowerShell restrictions, the reduced local admin rights, the rotated credentials. Recovery means returning to a state that is both clean and improved, not just clean.

3. **Reconnect in stages.** Bring the Victim back onto the lab network, but watch it before you trust it. Re-enable the network adapter, confirm the Wazuh agent reconnects and reports Active, and confirm Sysmon is logging again.
4. **Validate functionality.** Confirm the endpoint works: the account you need can log in, services run, the monitoring pipeline is intact. A recovered system that is not actually usable is not recovered.
5. **Monitor for reinfection.** For a set watch period, keep an eye on the Wazuh alerts for the Victim. Re-run a benign Atomic test (T1033) and confirm it still alerts, which proves your detection survived the restore. If the attacker's techniques reappear on their own, you missed something in Eradication and you loop back.
6. **Get sign-off.** In your plan, note who authorizes return to normal operations. In a solo lab that is you, but the habit of a formal go/no-go decision before resuming critical services is what the phase is teaching.

Intended result and why. The endpoint is restored from a trusted state, re-hardened, reconnected under observation, validated as working, and watched for reinfection, with an explicit authorization to resume. This matters because rushing recovery is how incidents reopen. Staged reconnection with monitoring catches a missed persistence mechanism before it becomes a second incident, and the formal sign-off forces someone to own the decision that the system is safe to use again.

Deliverable: a recovery validation report and a written authorization to resume normal operations.

Exercise 6: Lessons Learned

Phase goal: evaluate the response and turn it into lasting improvement. This is the phase that makes the next incident go better, and the one teams most often skip.

What you will do

1. **Run a post-incident review.** Walk the whole incident start to finish using your logs. What happened, when, and what you did at each step. Be honest about what went well and what did not.
2. **Determine root cause.** Not just "PowerShell ran malicious code," but why it was possible: unrestricted PowerShell, an over-privileged local account, and an entry vector. Separate the root cause from the contributing factors.

3. **Measure the response.** How long from first malicious action to detection? From detection to containment? These are your dwell time and response time. Compare them to your baseline from Preparation.
4. **Find the detection and response gaps.** Was there a technique in the attack chain that did not alert? A step in your plan that did not work? A tool you did not know how to use under pressure? Each gap is a concrete fix.
5. **Improve the controls and playbooks.** Turn each gap into an action: a new Wazuh rule, a tuned Sysmon config, a corrected step in your IR plan, a hardening baseline. Write a new detection rule for one technique that was weak, and confirm it fires.
6. **Update training and track actions to completion.** List every corrective action with an owner and a due date. An action nobody owns does not get done.
7. **Write the executive summary.** One page, plain language, no jargon: what happened, the impact, what you did, and what you are changing so it does not happen again. A leader should understand it without a security background.

Intended result and why. You produce a post-incident review with a clear root cause, measured response times, identified gaps, and a tracked corrective-action plan, plus at least one new detection you built and verified. This matters because this is where the lifecycle closes the loop: the improvements you make here become part of Preparation for the next incident. An organization that does Lessons Learned well gets measurably harder to breach over time; one that skips it keeps responding to the same incident forever.

Deliverable: a Post-Incident Review (PIR) and a Corrective Action Plan, plus the one-page executive summary.

Expected results, deliverables, and grading

By the end of the lab a student should be able to show a working detection pipeline, evidence they collected themselves, and the written artifacts from each phase. The table below is both a checklist for the student and a rubric for grading.

Phase	Deliverable	What "done" looks like
Preparation	One-page IR plan + detection check	Plan covers roles, severity, notification; three Atomic tests confirmed alerting
Identification	Incident log	Timeline with timestamps, IOC list, affected host and account, ATT&CK techniques, severity rating

Phase	Deliverable	What "done" looks like
Containment	Containment action log	Volatile evidence collected before isolation; host isolated; account disabled; persistence disabled; every action timestamped
Eradication	Eradication report	Persistence removed and verified, tooling deleted, credentials rotated, entry point hardened, validation results
Recovery	Recovery validation report	Restored from clean snapshot, re-hardened, staged reconnect, functionality validated, monitoring in place, sign-off
Lessons Learned	PIR + corrective action plan	Root cause, measured response times, gaps, tracked actions with owners, one new verified detection, executive summary

Grading suggestion (100 points). Each phase is worth up to 15 points for completeness and correctness of its deliverable, for 90 total. The final 10 points are for the quality of the executive summary and the one new detection rule the student built and verified, because those two prove the student can both communicate upward and improve the system, which is what separates an analyst from a responder.

Minimum pass: a student passes when the pipeline worked (they generated and detected the attack themselves), they collected real evidence with Velociraptor, and they produced a coherent incident log and post-incident review. The written artifacts matter as much as the technical steps; in real incident response, the work that is not documented did not happen.

Troubleshooting

Symptom	Likely cause	Fix
Wazuh dashboard will not load	Services still starting, or indexer unhealthy	Wait 2 to 3 minutes; check `systemctl is-active` for all four services; check cluster health returns green

Symptom	Likely cause	Fix
Installer fails on hardware check	Lab specs below the installer's defaults	Re-run with the `-i` flag to skip the check
Agent shows Disconnected	Victim cannot reach manager on TCP 1514, or wrong manager IP	`Test-NetConnection 192.168.56.10 -Port 1514`; confirm the `WAZUH_MANAGER` value; check host firewalls
No Sysmon events in Wazuh	Agent not reading the Sysmon channel, or Sysmon not installed	Confirm the `<localfile>` block is in ossec.conf; confirm Sysmon logs appear in Event Viewer; restart the wazuh service
Atomic test runs but no alert	Sysmon config too sparse, or rule level filtered out	Confirm you used the Wazuh sysmonconfig.xml; lower the alert-level filter in the dashboard to see level 3+
Velociraptor GUI unreachable	GUI bound to localhost, or wrong port	Confirm GUI bind address is 0.0.0.0 in server.config.yaml; browse to port 8889; accept the self-signed cert
Velociraptor client never appears	Client config points at wrong server address	Confirm the client config's server URL is the DFIR VM's lab IP; reinstall the client service
Mimikatz vanishes on download	Defender quarantined it (working as intended)	Add a temporary Defender exclusion for the Downloads folder, and note it for Lessons Learned
PowerShell download blocked	TLS or execution policy	Use `Set-ExecutionPolicy Bypass -Scope Process -Force`; ensure the build-time NAT adapter is connected

If a VM gets into a bad state during the exercises, that is what the `clean-build` snapshot is for. Roll back and start the affected part again.

Appendix: templates

Copy these into your own notes and fill them in as you work through the exercises.

Incident log (Identification)

Field	Entry
Incident ID / name	
Detected by	
Date and time detected	
Affected host(s)	
Affected account(s)	
ATT&CK techniques observed	
Indicators of compromise	
Timeline (time: event)	
Severity	

Containment action log

Time	Action taken	By whom	Reason	Evidence preserved?

Post-incident review (Lessons Learned)

Section	Notes
What happened (summary)	
Root cause	
Contributing factors	
Time to detect	
Time to contain	
What worked	

Section	Notes
What failed	
Detection / response gaps	
Corrective actions (owner, due date)	
Executive summary (one paragraph)	

References

- NIST SP 800-61 Rev. 3, Incident Response Recommendations and Considerations for Cybersecurity Risk Management
- Wazuh quickstart and installation guide
- Wazuh use case: monitoring Sysmon events
- Sysmon, Microsoft Sysinternals
- SwiftOnSecurity sysmon-config (alternative Sysmon config)
- Velociraptor releases, Velocidex
- Atomic Red Team, Red Canary
- MITRE ATT&CK

Software versions referenced in this guide: Wazuh 4.14, Velociraptor 0.77.3. Check each project's release page for the current version before you build; the commands change little, but always pull the latest stable release.

Cybersecurity Masterclass by Joshua Moses | CyberTrace Foundation.